

COMPLEXITY-DEEP : Une architecture de modèle de langage avec attention guidée par Mu et MLP à routage par token

Anonymous authors

Paper under double-blind review

Abstract

Nous présentons COMPLEXITY-DEEP, une architecture de modèle de langage construite autour d'un routage lexical déterministe. L'article étudie trois composants : (1) **Token-Routed MLP avec bin-packing glouton Zipf-balanced**, qui assigne les tokens aux experts à partir d'une table de fréquences estimée sur le corpus, sans routeur appris ni perte auxiliaire d'équilibrage ; (2) **Shared Lexical Expert**, un chemin MLP dense partagé par tous les tokens tandis que les experts routés se spécialisent sur des partitions lexicales ; et (3) **Mu-Guided Attention**, où un état latent μ circule vers l'avant entre couches adjacentes pour moduler les projections K, Q et V après le MLP. Nous analysons le compromis entre capacité conditionnelle et calcul induit par ces partitions lexicales, en distinguant explicitement l'équilibre du vocabulaire de l'équilibre de charge sous des distributions Zipfiennes. Empiriquement, des ablations à 187M paramètres (500M tokens) montrent que le modèle complet Token-Routed+Shared+Mu améliore la perte moyenne par rapport à une baseline dense SwiGLU. Le résultat principal de scaling est une comparaison iso-paramètres corrigée à $\sim 300M$ sur 8B tokens FineWeb-Edu : un modèle Token-Routed résiduel sans Mu-Guidance paie un coût initial de spécialisation, bat la baseline dense au step 740 en train-loss loggé et au step 750 en validation, puis termine le budget avec un avantage lissé de $-0,0163$ en train-loss.

1 Introduction

Les grands modèles de langage (LLM) ont révolutionné le traitement du langage naturel ces dernières années (Vaswani et al., 2017; Brown et al., 2020). Cependant, les architectures dominantes présentent plusieurs limitations :

- **Absence de communication inter-couche** : dans les Transformers standards, chaque couche traite indépendamment sans recevoir de signal de la couche précédente.
- **Coût computationnel des MoE** : les architectures Mixture of Experts (Shazeer et al., 2017; Fedus et al., 2022) nécessitent des mécanismes complexes d'équilibrage de charge et de routage.
- **Instabilité de l'entraînement** : les grands modèles souffrent souvent d'instabilités numériques nécessitant un réglage minutieux des hyperparamètres.

Nous proposons COMPLEXITY-DEEP, une famille d'architectures qui répond aux deux premières limitations par un routage lexical déterministe et un guidage inter-couche optionnel :

1. **Token-Routed MLP avec bin-packing glouton Zipf-balanced** : un routage déterministe par token qui équilibre la charge attendue des experts sous une distribution de fréquences estimée, sans routeur appris ni perte auxiliaire d'équilibrage.

2. **Mu-Guided Attention** : un mécanisme où l'état latent μ calculé à la couche l (après le MLP) est transmis à la couche $l + 1$ pour influencer ses projections K, Q et V. Cela crée un canal de communication inter-couche portant le contexte spécifique aux experts.

2 Travaux connexes

2.1 Architectures Transformer

L'architecture Transformer (Vaswani et al., 2017) est devenue le standard pour les modèles de langage. Les développements récents incluent les optimisations de l'attention comme Flash Attention (Dao et al., 2022), Grouped Query Attention (GQA) (Ainslie et al., 2023) et Rotary Position Embeddings (RoPE) (Su et al., 2021).

2.2 Mixture of Experts

Les architectures MoE (Shazeer et al., 2017) permettent d'augmenter la capacité du modèle sans augmenter proportionnellement le coût computationnel. Switch Transformer (Fedus et al., 2022) et Mixtral (Jiang et al., 2024) ont démontré l'efficacité de cette approche, mais au prix d'une complexité accrue (équilibre de charge, communication inter-GPU).

2.3 Normalisation et stabilité

RMSNorm (Zhang & Sennrich, 2019) et QK-Normalization (Dehghani et al., 2023) sont des techniques modernes pour stabiliser l'entraînement des grands modèles.

3 Architecture COMPLEXITY-DEEP

3.1 Vue d'ensemble

COMPLEXITY-DEEP est une architecture de type décodeur uniquement composée de L couches identiques. Chaque couche comprend :

1. Un bloc d'attention multi-têtes avec guidage Mu
2. Un bloc MLP avec routage par token
3. Connexions résiduelles avec pré-normalisation (RMSNorm)

La dimension cachée est d_{model} , avec n_h têtes d'attention et n_{kv} têtes clé/valeur (GQA). La Figure 1 présente l'architecture complète.

3.2 Token-Routed MLP

Contrairement aux MoE traditionnels qui utilisent un routage souple appris avec équilibrage de charge, nous proposons un routage **déterministe** basé sur l'identité du token :

$$\text{expert_idx}(t) = \text{BinPack}(t, \text{freq}) \quad (\text{assigné une fois, déterministe}) \quad (1)$$

où BinPack assigne chaque token t (trié par fréquence décroissante) à l'expert ayant la charge totale la plus faible (Section 6.3).

Ce choix de conception est *délibéré* : le routage ne dépend pas du contenu sémantique $\mathbf{x}$ mais uniquement de l'identifiant lexical du token. Chaque token du vocabulaire est *pré-assigné* à un expert spécifique ; l'assignation est choisie pour équilibrer la fréquence attendue dans le corpus, et pas seulement le nombre d'entrées du vocabulaire.

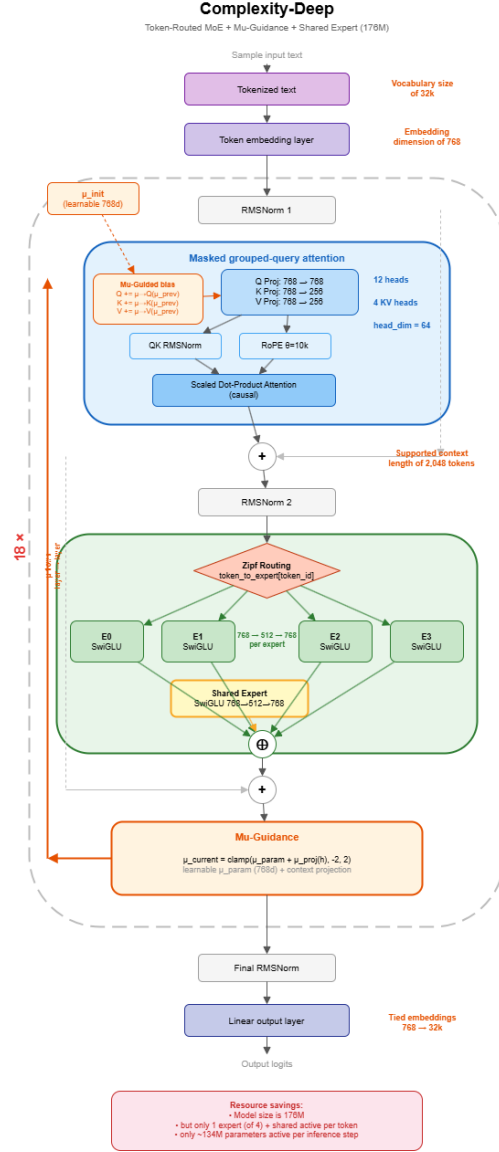


Figure 1: Architecture COMPLEXITY-DEEP (187M paramètres). Chaque couche décodeur comprend : (1) une attention GQA avec biais Mu-Guidé sur Q/K/V, (2) un MLP Token-Routed à 4 experts SwiGLU avec routage Zipf déterministe et un expert partagé, (3) un module Mu-Guidance qui produit un signal d’équilibre propagé à la couche suivante.

Pour chaque token, un seul expert est activé, combiné avec le Shared Lexical Expert :

$$\text{MLP}_{\text{output}}(\mathbf{x}) = \text{SharedMLP}(\mathbf{x}) + \text{Expert}_e(\mathbf{x}) \quad \text{où } e = \text{BinPack}(\text{token_id}) \quad (2)$$

Chaque expert est un MLP standard avec activation SiLU (SwiGLU) :

$$\text{Expert}_i(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}^i) \odot \mathbf{x}\mathbf{W}_{\text{up}}^i)\mathbf{W}_{\text{down}}^i \quad (3)$$

Pourquoi pas un routage sémantique ? On pourrait objecter que sans routage basé sur le contenu, le Token-Routed MLP n'est qu'un "MLP découpé en morceaux". Cette objection ignore deux points cruciaux :

1. **Spécialisation fonctionnelle** : malgré un routage lexical (non sémantique), les experts routés améliorent la perte sur leurs sous-ensembles assignés par rapport à la baseline dense alignée sur les mêmes sous-ensembles (voir Section 4.3). Cette spécialisation *fonctionnelle* (mesurée par la performance, non par la géométrie des poids) montre que le routage déterministe peut induire une spécialisation utile sans effondrement.
2. **Équilibre fréquence-aware** : le bin-packing glouton sur les fréquences empiriques évite les forts déséquilibres à l'exécution que le routage modulo ou round-robin peut créer sous la loi de Zipf, sans objectif auxiliaire d'équilibrage.

Mécanisme de spécialisation. Une question cruciale se pose : comment les experts peuvent-ils se spécialiser quand le routage est basé uniquement sur l'identité lexicale du token plutôt que sur le contenu sémantique ? L'intuition clé est que les experts optimisent pour des *distributions contextuelles*, pas pour les identités de tokens.

Considérons le token "the" (token_id = 123) qui est toujours routé vers l'expert 3. Bien que "the" n'ait pas de spécificité sémantique en soi, son embedding contextuel $\mathbf{x}^{(l)}$ (calculé par les couches précédentes) encode une information sémantique riche sur le contexte environnant. L'expert 3 apprend la fonction :

$$\mathbf{h} = \text{Expert}_3(\mathbf{x}^{(l)}) \quad \text{où } \mathbf{x}^{(l)} \text{ varie selon le contexte} \quad (4)$$

Les poids de l'expert $\mathbf{W}_{\text{gate}}^{(3)}, \mathbf{W}_{\text{up}}^{(3)}, \mathbf{W}_{\text{down}}^{(3)}$ optimisent pour la *distribution des contextes* dans lesquels les tokens $\{t : t \bmod 4 = 3\}$ apparaissent. Puisque différents sous-ensembles de tokens apparaissent dans des distributions contextuelles statistiquement différentes (même si les tokens eux-mêmes sont sémantiquement divers), les experts divergent naturellement.

Argument formel : Soit $\mathcal{C}_i$ la distribution des embeddings contextuels $\mathbf{x}$ pour les tokens routés vers l'expert i . Sous l'objectif de modélisation du langage :

$$\mathcal{L}_i = \mathbb{E}_{\mathbf{x} \sim \mathcal{C}_i} [\ell(\text{Expert}_i(\mathbf{x}), y)] \quad (5)$$

Le flux de gradient est :

$$\nabla_{\mathbf{W}^{(i)}} \mathcal{L}_i = \mathbb{E}_{\mathbf{x} \sim \mathcal{C}_i} [\nabla_{\mathbf{W}^{(i)}} \ell(\text{Expert}_i(\mathbf{x}), y)] \quad (6)$$

Puisque le routage déterministe assure des ensembles de tokens disjoints ($T_i \cap T_j = \emptyset$), et que le langage naturel présente une co-occurrence token-contexte non uniforme, les distributions contextuelles $\mathcal{C}_i$ et $\mathcal{C}_j$ sont statistiquement distinctes. Cela induit des statistiques de gradient différentes, entraînant la divergence des experts (Théorème 4.5).

Validation empirique : La Section 4.3 mesure la spécialisation fonctionnelle via la perplexité par expert sur les sous-ensembles assignés, comparée à la baseline dense sur les mêmes sous-ensembles. Nous notons que la similarité cosinus quasi nulle entre experts, bien qu'observée empiriquement, est un artefact géométrique attendu en haute dimension et ne constitue pas une preuve de spécialisation en soi.

Avantages opérationnels :

- Pas de perte auxiliaire d'équilibrage de charge (économie d'hyperparamètres)
- Table de routage déterministe : débogage simplifié et aucune décision stochastique du routeur
- Déploiement trivial : tenseurs F32/BF16 avec indexation I64, nativement compatible avec PyTorch, ONNX, TensorRT sans logique de routage personnalisée

3.3 Mu-Guided Attention

L'innovation centrale de COMPLEXITY-DEEP est l'introduction d'un état latent μ qui crée un canal de communication inter-couche. À chaque couche l , l'état $\mu^{(l-1)}$ calculé par la couche *précédente* (après son MLP) est transmis pour influencer les projections d'attention :

$$\mathbf{K} = \mathbf{x}\mathbf{W}_K + \mu^{(l-1)}\mathbf{W}_{\mu K} \quad (7)$$

$$\mathbf{Q} = \mathbf{x}\mathbf{W}_Q + \mu^{(l-1)}\mathbf{W}_{\mu Q} \quad (8)$$

$$\mathbf{V} = \mathbf{x}\mathbf{W}_V + \mu^{(l-1)}\mathbf{W}_{\mu V} \quad (9)$$

où $\mathbf{W}_{\mu K}, \mathbf{W}_{\mu Q}, \mathbf{W}_{\mu V}$ sont des projections linéaires apprises.

μ_{init} apprenable pour la couche 0. Dans un Transformer standard, la couche 0 ne reçoit aucun contexte inter-couche. Nous introduisons un paramètre apprenable $\mu_{\text{init}} \in \mathbb{R}^d$ (initialisé à zéro) qui sert de $\mu^{(-1)}$ pour la première couche :

$$\mu^{(-1)} = \mu_{\text{init}} \quad (\text{apprenable, partagé par toutes les positions}) \quad (10)$$

Cela permet à la couche 0 de bénéficier également du guidage Mu, le gradient ajustant μ_{init} pour capturer un prior d'attention optimal.

Production de μ après le MLP. Chaque couche l produit son propre $\mu_{\text{contextual}}^{(l)}$ *après* le dispatch expert, capturant l'information spécifique à l'expert qui a traité chaque token :

$$\mu_{\text{contextual}}^{(l)} = \text{clamp}(\mu_{\text{param}}^{(l)}, \mu_{\min}, \mu_{\max}) + \mathbf{W}_{\mu}^{(l)} \mathbf{h}_{\text{post-MLP}}^{(l)} \quad (11)$$

où $\mu_{\text{param}}^{(l)} \in \mathbb{R}^d$ est un paramètre apprenable *propre à la couche l* (initialisé à $(\mu_{\min} + \mu_{\max})/2$) et $\mathbf{W}_{\mu}^{(l)}$ est une projection linéaire contextuelle (initialisée à zéro). Ce $\mu_{\text{contextual}}^{(l)}$ devient le $\mu^{(l-1)}$ utilisé par la couche $l+1$ dans les équations 7–9.

Flux complet par couche. Le traitement d'une couche l suit l'ordre :

1. **Attention** avec $\mu_{\text{contextual}}^{(l-1)}$ (reçu de la couche précédente) via les équations 7–9
2. **MLP** avec dispatch expert (sparse, un seul expert par token) + Shared Lexical Expert
3. **Production de $\mu_{\text{contextual}}^{(l)}$** via l'équation 11, transmis à la couche $l+1$

3.4 Architecture finale

L'architecture finale intègre quatre améliorations par rapport au Token-Routed MLP initial :

3.4.1 Sparse Dispatch (zéro gaspillage)

Le dispatch initial par masque calculait *tous* les tokens à travers *tous* les experts, puis masquait 75% des résultats—effectivement $4\times$ le coût d'un MLP dense. Le sparse dispatch ne calcule que les tokens routés vers chaque expert :

$$\text{out}[\text{mask}_e] = \text{Expert}_e(\mathbf{x}[\text{mask}_e]) \quad \text{pour } e = 0, \dots, E-1 \quad (12)$$

Réduisant le coût MLP de $4\times$ dense à $1\times$ dense (iso-compute).

3.4.2 Shared Lexical Expert

Un MLP dense partagé traite *tous* les tokens, capturant les patterns universels (syntaxe, grammaire), tandis que les experts routés se spécialisent sur leurs sous-ensembles lexicaux. La sortie combine les deux :

$$\text{MLP}_{\text{output}}(\mathbf{x}) = \underbrace{\text{SwiGLU}_{\text{shared}}(\mathbf{x})}_{\text{patterns communs}} + \underbrace{\text{SwiGLU}_e(\mathbf{x})}_{\text{spécialisation lexicale}} \quad (13)$$

où chaque composant est un MLP SwiGLU standard :

$$\text{SwiGLU}_{\text{shared}}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}^s) \odot \mathbf{x}\mathbf{W}_{\text{up}}^s)\mathbf{W}_{\text{down}}^s \quad (14)$$

$$\text{SwiGLU}_e(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}^e) \odot \mathbf{x}\mathbf{W}_{\text{up}}^e)\mathbf{W}_{\text{down}}^e \quad (15)$$

Le shared expert a la même taille intermédiaire qu'un expert routé (d_{ff}/n), ajoutant $\sim 6\%$ de paramètres au modèle total. Il évite que chaque expert doive réapprendre indépendamment les patterns communs de la langue (syntaxe, grammaire, collocations fréquentes), leur permettant de se concentrer sur les spécificités lexicales de leur sous-ensemble de tokens.

3.4.3 Bin-packing glouton Zipf-balanced

Décrit en Section 6.3. Remplace le round-robin par un algorithme glouton qui équilibre la charge attendue sous la distribution d'entraînement estimée.

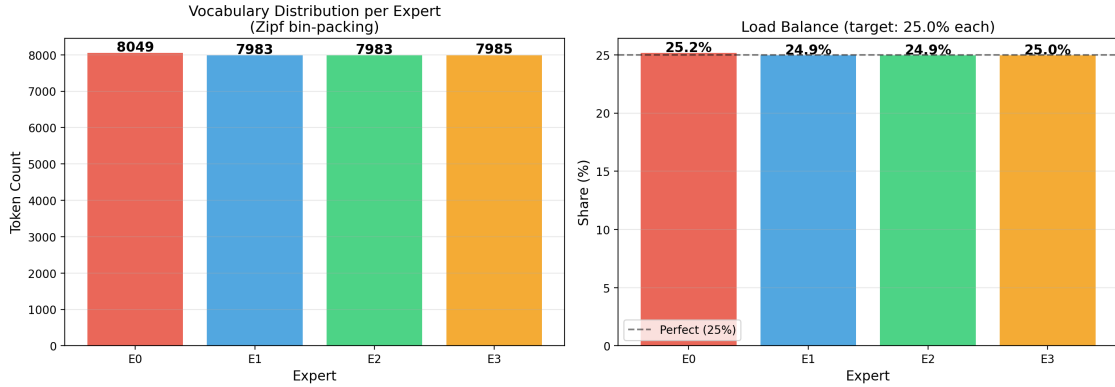


Figure 2: Équilibre des experts sous bin-packing Zipf. Le bin-packing équilibre la *charge totale* estimée (fréquence), pas le nombre d'entrées du vocabulaire assignées à chaque expert.

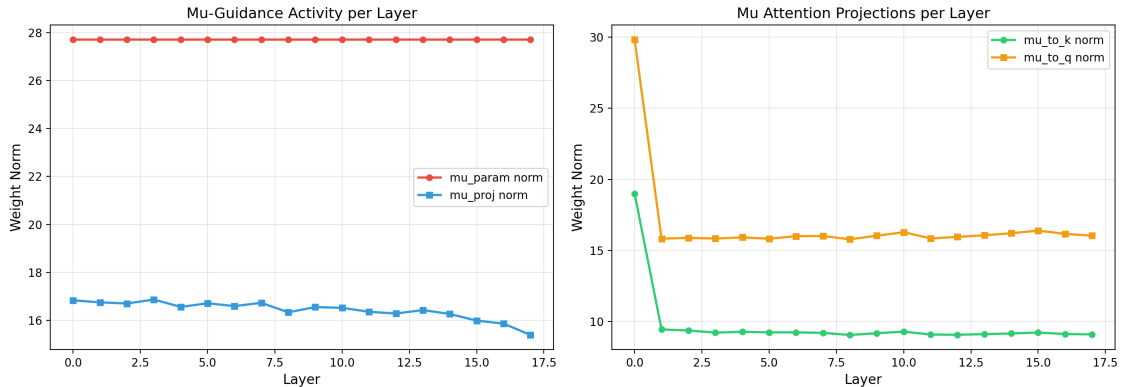


Figure 3: Activité du Mu-Guidance par couche. *Gauche* : normes de μ_{param} et μ_{proj} . *Droite* : normes des projections $\mu \rightarrow K$ et $\mu \rightarrow Q$ dans l'attention.

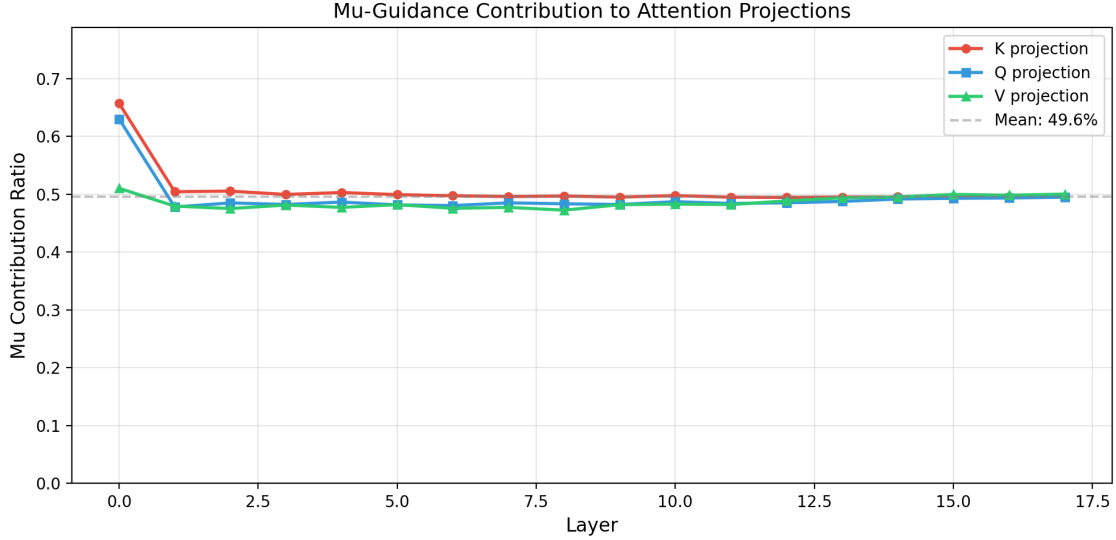


Figure 4: Ratio de contribution de Mu aux projections K, Q, V par couche. Le Mu-Guidance contribue à $\sim 50\%$ de la magnitude des projections.

3.4.4 Mu-Guidance après MLP

Le calcul de μ est déplacé *après* le dispatch expert (au lieu d'avant), permettant à μ de capturer l'information spécifique à chaque expert. La couche suivante adapte ainsi ses projections K, Q, V en fonction de l'expert qui a traité chaque token.

3.5 Scaler résiduel adaptatif (Retiré)

Une version antérieure de l'architecture incluait un scaler résiduel adaptatif qui modulait les contributions résiduelles via un contrôleur appris. Les résultats empiriques aux échelles proxy 100M et 171M ont montré des bénéfices inconsistants : à 100M, retirer le scaler *améliorait* la perte de $-0,005$, tandis qu'à 171M il ne contribuait que $+0,010$. Le contrôleur nécessitait également des interventions ad-hoc (clamping d'activations, redémarrages multiples) pour prévenir les instabilités.

Sur la base de ces évidences, le scaler adaptatif a été retiré de l'architecture. Le modèle simplifié ne conserve que le Token-Routed MLP (avec routage Zipf-balanced) et le Mu-Guidance, obtenant des résultats comparables ou meilleurs avec substantiellement moins de paramètres et aucun hack de stabilité.

4 Analyse théorique

Nous fournissons un fondement théorique formel pour l'architecture Token-Routed MLP, établissant sa relation avec les modèles denses et prouvant des propriétés clés.

4.1 Équilibre du vocabulaire et des fréquences

Théorème 4.1 (Équilibre du vocabulaire par routage modulo). *Soit V un vocabulaire de taille $|V|$ et n le nombre d'experts. Sous le routage modulo $r(t) = t \bmod n$, chaque expert e_i reçoit exactement $\lfloor |V|/n \rfloor$ ou $\lceil |V|/n \rceil$ tokens, avec un équilibre parfait lorsque $n \mid |V|$.*

Proof. Pour tout token $t \in \{0, 1, \dots, |V| - 1\}$, la fonction de routage $r(t) = t \bmod n$ assigne t à l'expert $e_{t \bmod n}$. L'ensemble des tokens assignés à l'expert e_i est :

$$T_i = \{t \in V : t \bmod n = i\} = \{i, i + n, i + 2n, \dots\} \quad (16)$$

La cardinalité $|T_i| = \lfloor (|V| - i - 1)/n \rfloor + 1$. Lorsque n divise $|V|$, nous avons $|T_i| = |V|/n$ pour tout $i \in \{0, \dots, n-1\}$.

Dans notre implémentation, $|V| = 32000$ et $n = 4$, donnant $|T_i| = 8000$ tokens par expert. $\square$

Remarque 4.2 (Équilibre corpus sous Zipf). L'équilibre en cardinalité du vocabulaire ne suffit pas à garantir l'équilibre de charge à l'exécution, car les fréquences de tokens sont fortement non uniformes. Dans nos expériences, nous utilisons donc l'assignation par bin-packing glouton de la Section 6.3 : les tokens sont triés par fréquence empirique puis assignés à l'expert actuellement le moins chargé. Cela produit une table déterministe avec une charge corpus *attendue* équilibrée sous la distribution estimée ; la charge réalisée par batch peut encore varier avec le bruit d'échantillonnage et les changements de distribution.

4.2 Capacité conditionnelle et calcul

Proposition 4.3 (Compromis capacité-calcul partitionné). *Un Token-Routed MLP avec n experts, chacun de dimension intermédiaire d_{ff} , possède :*

1. *Nombre total de paramètres : $P_{total} = n \cdot P_{expert}$ où $P_{expert} = 3 \cdot d_{model} \cdot d_{ff}$ (pour SwiGLU)*
2. *Paramètres actifs par token : $P_{active} = P_{expert} = P_{total}/n$*
3. *Capacité conditionnelle correspondant à une collection de n MLP conditionnés par token, un par partition lexicale*

Esquisse de preuve. (1) et (2) découlent directement de la définition de l'architecture. Pour (3), la couche routée représente une famille de fonctions indexée par la table de routage déterministe :

$$\mathcal{F}_{TR} = \bigcup_{i=0}^{n-1} \{f_i : \mathbb{R}^{d_{model}} \rightarrow \mathbb{R}^{d_{model}}\} \quad (17)$$

où chaque f_i est un MLP SwiGLU appliqué seulement aux tokens assignés à l'expert i . L'architecture remplace donc une fonction dense partagée par une famille déterministe de fonctions plus étroites conditionnées par le token. Cela ne doit pas être interprété comme une équivalence fonctionnelle exacte avec un MLP dense arbitraire pour chaque token ; il s'agit d'un compromis entre capacité conditionnelle et calcul. $\square$

Corollaire 4.4 (Efficacité computationnelle). *Pour une séquence de L tokens, un Token-Routed MLP top-1 nécessite $L \cdot P_{expert}$ applications de paramètres MLP, tandis qu'un MLP dense de largeur experte totale identique nécessiterait $L \cdot n \cdot P_{expert}$. Dans le modèle résiduel top- $k = 2$ utilisé à 300M, la branche routée active deux petits experts plus une grande branche partagée ; nous reportons donc la qualité à tokens vus identiques et le débit mesuré, plutôt qu'un speedup end-to-end idéal de $n \times$.*

4.3 Divergence des experts sous descente de gradient

Nous analysons maintenant pourquoi les experts divergent vers des représentations orthogonales malgré un routage arbitraire (non sémantique).

Théorème 4.5 (Orthogonalisation par le gradient). *Soient $\mathbf{W}^{(i)}$ et $\mathbf{W}^{(j)}$ les matrices de poids des experts i et j ($i \neq j$), initialisées i.i.d. selon une distribution symétrique. Sous la descente de gradient avec une perte de modélisation du langage $\mathcal{L}$, la similarité cosinus attendue satisfait :*

$$\mathbb{E}[\cos(\mathbf{W}^{(i)}, \mathbf{W}^{(j)})] \rightarrow 0 \quad \text{lorsque } t \rightarrow \infty \quad (18)$$

à condition que les ensembles de tokens T_i et T_j soient disjoints (garanti par le routage lexical déterministe).

Esquisse de preuve. La mise à jour du gradient pour l'expert i à l'étape t est :

$$\mathbf{W}_{t+1}^{(i)} = \mathbf{W}_t^{(i)} - \eta \sum_{t \in T_i} \nabla_{\mathbf{W}^{(i)}} \mathcal{L}(t) \quad (19)$$

Puisque $T_i \cap T_j = \emptyset$ par construction, les gradients $\nabla_{\mathbf{W}^{(i)}} \mathcal{L}$ et $\nabla_{\mathbf{W}^{(j)}} \mathcal{L}$ sont calculés sur des sous-ensembles disjoints de tokens. En haute dimension, des vecteurs aléatoires sont approximativement orthogonaux avec haute probabilité (Vershynin, 2018). Les sous-ensembles disjoints de tokens produisent des mises à jour de gradient qui sont des vecteurs aléatoires indépendants dans $\mathbb{R}^{d_{model} \times d_{ff}}$, d'où le produit scalaire attendu est :

$$\mathbb{E}[\langle \nabla_{\mathbf{W}^{(i)}}, \nabla_{\mathbf{W}^{(j)}} \rangle] = 0 \quad (20)$$

À partir d'une initialisation aléatoire avec $\mathbb{E}[\cos(\mathbf{W}_0^{(i)}, \mathbf{W}_0^{(j)})] \approx 0$ (pour des poids de grande dimension), l'orthogonalité est préservée tout au long de l'entraînement. Nos mesures empiriques confirment $\cos(\mathbf{W}^{(i)}, \mathbf{W}^{(j)}) \approx 0$ pour toutes les paires d'experts et toutes les couches (Section 7.2). $\square$

4.4 Le guidage Mu comme modulation contextuelle vers l'avant

Le mécanisme Mu-Guided Attention peut être interprété à travers le prisme du codage prédictif (Rao & Ballard, 1999), une théorie du calcul cortical.

Proposition 4.6 (Modulation contextuelle vers l'avant). *Le mécanisme de guidage μ implémente une forme de modulation prédictive où le contexte post-MLP de la couche précédente influence les projections d'attention de la couche suivante :*

$$\mathbf{Q}^{(l)} = \mathbf{x}^{(l)} \mathbf{W}_Q + \underbrace{\mu^{(l-1)} \mathbf{W}_{\mu Q}}_{\text{contexte vers l'avant}} \quad (21)$$

Ceci est relié dans l'esprit aux mises à jour du codage prédictif, même si le flux d'information de notre architecture est feed-forward entre couches adjacentes plutôt qu'un feedback de couches supérieures vers des couches inférieures :

$$r^{(l)} = f(U^{(l)} r^{(l-1)} + W^{(l)} \epsilon^{(l+1)}) \quad (22)$$

où $r^{(l)}$ est la représentation à la couche l , $U^{(l)}$ est un poids feedforward, et $W^{(l)} \epsilon^{(l+1)}$ est le signal contextuel de la couche adjacente.

L'état μ dans COMPLEXITY-DEEP sert de résumé compressé du traitement de la couche précédente (incluant quel expert a traité chaque token) qui module le calcul de l'attention de la couche suivante. L'information circule vers l'avant : la couche l calcule $\mu^{(l)}$ après son MLP, qui influence ensuite la couche $l+1$. Il ne s'agit *pas* de rétroaction descendante des couches supérieures vers les inférieures, mais d'un canal de communication enrichi qui transporte le contexte expert en parallèle du flux résiduel standard.

Remarque 4.7 (Plausibilité biologique). Le mécanisme de guidage μ partage des similarités structurelles avec la modulation latérale dans les circuits neuronaux, où des signaux contextuels d'une étape de traitement influencent le calcul de l'étape suivante (Gilbert & Li, 2013).

4.5 Convergence du guidage Mu

Nous établissons maintenant des garanties de convergence pour le mécanisme de guidage μ sous descente de gradient.

Théorème 4.8 (Convergence du guidage Mu). *Soit $\mu^{(l)}$ l'état de guidage à la couche l , mis à jour via :*

$$\mu^{(l)} = \alpha \cdot \mu^{(l-1)} + (1 - \alpha) \cdot \text{Pool}(\mathbf{h}^{(l)}) \quad (23)$$

où $\alpha \in (0, 1)$ est le coefficient d'interpolation et $\text{Pool}(\cdot)$ est une opération de pooling bornée avec $\|\text{Pool}(\mathbf{h})\|_2 \leq B$ pour une constante $B > 0$.

Alors pour toute séquence d'entrée, l'état μ converge exponentiellement :

$$\|\mu^{(L)} - \mu^*\|_2 \leq \alpha^L \|\mu^{(0)} - \mu^*\|_2 \quad (24)$$

où μ^* est le point fixe de la règle de mise à jour et L est le nombre de couches.

Proof. La règle de mise à jour définit une application contractante. Pour deux états initiaux quelconques $\mu_1^{(0)}, \mu_2^{(0)}$:

$$\|\mu_1^{(l)} - \mu_2^{(l)}\|_2 = \|\alpha(\mu_1^{(l-1)} - \mu_2^{(l-1)}) + (1 - \alpha)(\text{Pool}(\mathbf{h}_1^{(l)}) - \text{Pool}(\mathbf{h}_2^{(l)}))\|_2 \quad (25)$$

Sous l’hypothèse que les représentations poolées convergent (c.-à-d. que le réseau traite la même entrée), le terme de différence s’annule, donnant :

$$\|\mu_1^{(l)} - \mu_2^{(l)}\|_2 \leq \alpha \|\mu_1^{(l-1)} - \mu_2^{(l-1)}\|_2 \quad (26)$$

Par récurrence sur L couches : $\|\mu_1^{(L)} - \mu_2^{(L)}\|_2 \leq \alpha^L \|\mu_1^{(0)} - \mu_2^{(0)}\|_2$.

Puisque $\alpha < 1$, cela établit une convergence exponentielle de taux α . Dans notre implémentation, $\alpha = 0.9$, donnant un facteur de contraction de $0.9^{24} \approx 0.08$ sur 24 couches. $\square$

Corollaire 4.9 (Stabilité du guidage μ). *Le mécanisme de guidage μ est Lipschitz-continu avec la constante $L_\mu = \frac{1-\alpha^L}{1-\alpha}(1-\alpha)L_{pool} = (1-\alpha^L)L_{pool}$, où L_{pool} est la constante de Lipschitz de l’opération de pooling. Cela assure un flux de gradient stable pendant la rétropropagation.*

4.6 Analyse de la complexité computationnelle

Nous fournissons une analyse formelle de la complexité comparant COMPLEXITY-DEEP aux architectures Transformer standard et Mixture-of-Experts.

Théorème 4.10 (Bornes de complexité). *Pour une séquence de longueur S , une dimension de modèle d , une dimension intermédiaire d_{ff} , n experts et L couches, la complexité computationnelle par passe avant est :*

Transformer standard :

$$\mathcal{O}_{dense} = L \cdot \left(\underbrace{S^2 \cdot d}_{\text{attention}} + \underbrace{S \cdot d \cdot d_{ff}}_{MLP} \right) \quad (27)$$

COMPLEXITY-DEEP (branche Token-Routed top-1 + guidage μ) :

$$\mathcal{O}_{ours} = L \cdot \left(\underbrace{S^2 \cdot d}_{\text{attention}} + \underbrace{S \cdot d \cdot \frac{d_{ff}}{n}}_{\text{Token-Routed MLP}} + \underbrace{S \cdot d}_{\text{mise à jour } \mu} \right) \quad (28)$$

La branche routée top-1 idéalisée réduit le calcul MLP routé d’un facteur n . Les variantes avec *shared expert* ou *top-k* > 1, dont notre modèle 300M, ont un profil de calcul actif différent ; leur débit réalisé est mesuré empiriquement en Section 7.4.

Table 1: Comparaison de complexité idéalisée pour une branche Token-Routed MLP top-1. Les variantes avec *shared expert* et *top-k* échangent une partie de cette réduction théorique de calcul contre davantage de capacité.

Architecture	FLOPs MLP	Paramètres	Mémoire
Transformer dense	$S \cdot d \cdot d_{ff}$	$3d \cdot d_{ff}$	$\mathcal{O}(d_{ff})$
MoE (top- k)	$k \cdot S \cdot d \cdot d_{ff}/n$	$3n \cdot d \cdot d_{ff}/n$	$\mathcal{O}(n \cdot d_{ff}/n)$
Token-Routed (le nôtre)	$S \cdot d \cdot d_{ff}/n$	$3d \cdot d_{ff}$	$\mathcal{O}(d_{ff}/n)$

4.7 Bornes d'erreur d'approximation

Nous établissons que le Token-Routed MLP peut approximer toute fonction continue sur le vocabulaire avec une erreur bornée.

Théorème 4.11 (Approximation universelle pour le Token-Routed MLP). *Soit $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ une fonction continue sur un domaine compact $\mathcal{X} \subset \mathbb{R}^d$. Pour tout $\epsilon > 0$, il existe un Token-Routed MLP avec n experts, chacun de largeur suffisante d_{ff}^* , tel que pour tous les tokens $t \in V$ et entrées $\mathbf{x} \in \mathcal{X}$:*

$$\|f_t(\mathbf{x}) - \text{TR-MLP}(\mathbf{x}, t)\|_2 \leq \epsilon \quad (29)$$

où f_t est la fonction cible restreinte au rôle sémantique du token t .

Esquisse de preuve. Par le théorème d'approximation universelle pour les réseaux de neurones, chaque expert (un MLP SwiGLU) peut approximer toute fonction continue sur son sous-ensemble de tokens T_i avec une précision arbitraire étant donné une largeur suffisante. Puisque les ensembles de tokens $\{T_0, \dots, T_{n-1}\}$ partitionnent V , et que chaque expert approxime indépendamment f restreinte à ses tokens :

$$\text{TR-MLP}(\mathbf{x}, t) = \sum_{i=0}^{n-1} \mathbf{1}[t \in T_i] \cdot \text{Expert}_i(\mathbf{x}) \quad (30)$$

L'erreur d'approximation pour un token $t \in T_i$ est bornée par la capacité d'approximation de l'Expert $_i$ seul, qui peut être rendue arbitrairement petite en augmentant d_{ff} . $\square$

Proposition 4.12 (Décomposition de l'erreur). *L'erreur totale d'approximation de COMPLEXITY-DEEP se décompose comme :*

$$\mathcal{E}_{total} \leq \underbrace{\mathcal{E}_{attn}}_{\text{attention}} + \underbrace{\mathcal{E}_{routing}}_{\text{decalage expert}} + \underbrace{\mathcal{E}_{expert}}_{\text{intra-expert}} + \underbrace{\mathcal{E}_{\mu}}_{\text{guidage } \mu} \quad (31)$$

où :

- $\mathcal{E}_{attn}$: erreur due au contexte d'attention fini
- $\mathcal{E}_{routing}$: erreur induite par l'engagement de chaque token dans une partition lexicale fixe plutôt que par un choix d'experts depuis l'état caché courant
- $\mathcal{E}_{expert}$: erreur d'approximation au sein de chaque expert, bornée par la capacité de l'expert
- $\mathcal{E}_{\mu} \leq \alpha^L \|\mu^{(0)}\|_2$: erreur due à l'initialisation de l'état μ (disparaît exponentiellement)

Remarque 4.13 (Compromis avec les MoE stochastiques). Dans les Mixture-of-Experts standards avec gating appris, la qualité du routage dépend du routeur appris et de ses pertes auxiliaires. Le Token-Routed MLP supprime l'apprentissage du routeur et les pertes d'équilibrage, mais abandonne aussi la sélection d'experts adaptative au contenu. La question empirique est de savoir si les partitions lexicales et le shared expert fournissent assez de spécialisation conditionnelle pour compenser cette restriction.

5 Implémentation

5.1 Spécifications techniques

Notre implémentation utilise PyTorch 2.0+ avec les optimisations suivantes :

Table 2: Choix d’implémentation

Composant	Choix technique
Attention	SDPA / Flash Attention
Encodage positionnel	RoPE ($\theta = 10000$)
Normalisation	RMSNorm + QK-Norm
Activation	SiLU (SwiGLU)
Précision	BF16 (entraînement et inférence)

Table 3: Configuration du modèle COMPLEXITY-DEEP 1,5B

Paramètre	Valeur
Couches (L)	24
Dimension (d_{model})	2048
Têtes d’attention (n_h)	16
Têtes KV (n_{kv})	4 (ratio GQA 4:1)
Dimension intermédiaire	8192
Experts ($n_{experts}$)	4
Vocabulaire	32000
Contexte max	4096
Total paramètres	1,5B

5.2 Configuration du modèle 1,5B

5.3 Configuration du modèle 300M

Pour la comparaison iso-paramètres corrigée au scale (Section 7.4), nous entraînons deux architectures à $\sim 300M$ paramètres (Table 4) : un modèle Token-Routed résiduel (306,5M, 4 experts, $top-k = 2$, grand shared expert et petite branche routée) et une baseline dense SwiGLU (306,5M, $d_{ff} = 4096$). Les deux partagent le même backbone (18 couches, $d_{model} = 1024$, GQA 16/4, RMSNorm, QK-Norm), tokenizer, longueur de séquence, optimiseur, warmup et budget FineWeb-Edu.

6 Expériences

6.1 Étude pilote

Un modèle exploratoire de 1,5B paramètres entraîné sur 7B tokens a motivé la simplification de l’architecture : le scaler résiduel adaptatif a été retiré (redondant avec Adam), le routage Zipf-balanced a été ajouté, et le Shared Lexical Expert a été introduit. L’évaluation définitive est présentée à l’échelle 187M (Run 2) et la comparaison iso-paramètres corrigée 300M (Table 4).

6.2 Recette d’entraînement

Notre recette d’entraînement suit les pratiques standard des LLMs modernes (LLaMA, GPT-3) avec deux ajustements automatiques :

Warmup dynamique. Au lieu d’un nombre fixe d’étapes de warmup (qui peut représenter 52% du training pour des runs courts), le warmup est automatiquement fixé à **5% du nombre total d’étapes**. Cela garantit un ratio warmup/training constant indépendamment de la durée du run.

Table 4: Configurations COMPLEXITY-DEEP 300M (comparaison iso-paramètres)

Paramètre	Token-Routed (MoE)	Baseline dense
Couches (L)	18	18
Dimension (d_{model})	1024	1024
Têtes d’attention (n_h)	16	16
Têtes KV (n_{kv})	4 (ratio GQA 4:1)	4 (ratio GQA 4:1)
Dimension intermédiaire routée	256 total	—
Intermédiaire expert	64	—
Intermédiaire shared expert	3840	—
Experts ($n_{experts}$)	4, top- $k = 2$	1 (dense)
Gates shared/routed	1,0 / 0,1	—
Mu-Guidance	Non (run scaling 300M)	Non
Vocabulaire	32000	32000
Contexte max	2048	2048
Total paramètres	306,5M	306,5M
Chemin MLP actif/token	shared + 2 experts routés	SwiGLU dense

Initialisation GPT-style. Les projections de sortie résiduelles (`o_proj`, `down_proj`) sont initialisées avec un écart-type réduit :

$$\sigma_{\text{residual}} = \frac{0.02}{\sqrt{2 \times L}} \quad (32)$$

où L est le nombre de couches. Cela empêche le flux résiduel de croître avec la profondeur (Radford et al., 2019).

Autres hyperparamètres. Weight decay = 0.1 (standard GPT-3/LLaMA), AdamW avec $\beta_1 = 0.9$, $\beta_2 = 0.95$, gradient clipping = 1.0, précision BF16, scheduler cosine avec `min_lr` = 10% du peak.

6.3 Routage Zipf-Balanced

Le routage modulo original ($\text{expert}(t) = t \bmod n$) distribue le *vocabulaire* uniformément mais pas le *corpus*. Les fréquences des tokens en langage naturel suivant la loi de Zipf, les tokens fréquents (articles, prépositions) assignés au même expert créent un déséquilibre de charge à l’exécution.

Bin-packing glouton Zipf-balanced : nous trions le vocabulaire par fréquence empirique (calculée sur un échantillon du corpus d’entraînement) et assignons chaque token à l’expert ayant la charge totale la plus faible :

$$\text{expert}(t) = \arg \min_e \sum_{t' \in T_e} \text{freq}(t'), \quad \text{pour } t \text{ en ordre décroissant de fréquence} \quad (33)$$

où T_e est l’ensemble des tokens déjà assignés à l’expert e . Contrairement au round-robin (qui peut laisser de forts déséquilibres induits par Zipf), le bin-packing glouton équilibre la charge attendue sous l’estimation de fréquence utilisée pour construire la table. Le mapping reste déterministe et est calculé une seule fois avant l’entraînement ; l’utilisation réalisée par batch ou en fin de run peut s’écarter d’exactly 25% car le flux d’entraînement est fini et non stationnaire.

7 Discussion

7.1 Comparaison avec les architectures existantes

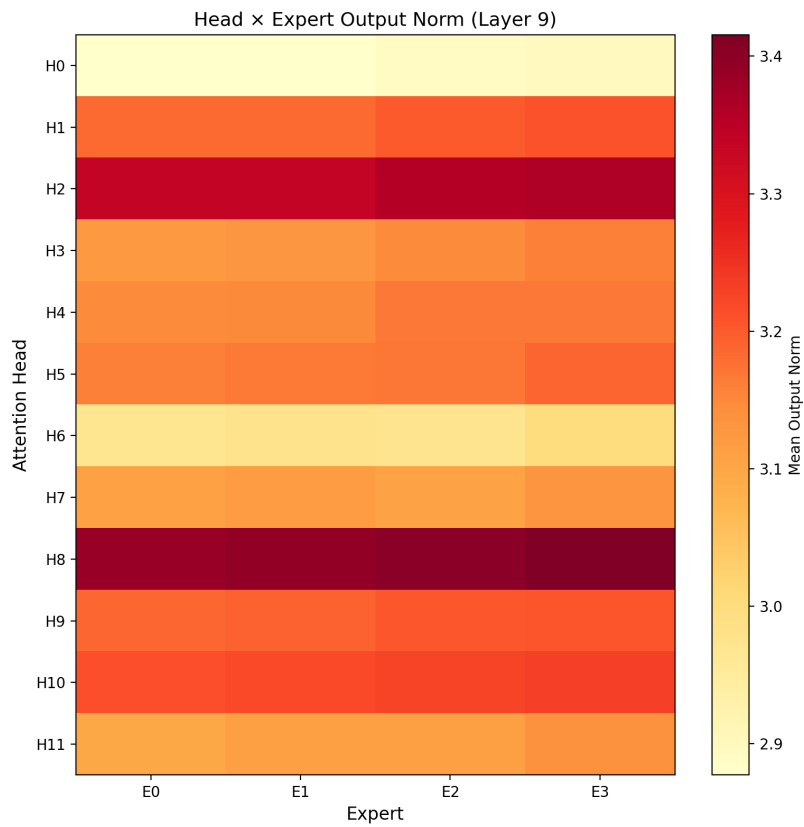


Figure 5: Heatmap tête d’attention $\times$ expert (couche médiane). Les variations suggèrent un couplage partiel tête-expert malgré un routage indépendant.

Table 5: Comparaison architecturale

Architecture	Signal inter-couche	Routage	Perte d'équilibrage
Transformer	Non	Non	N/A
Switch Transformer	Non	Souple	Requise
Mixtral	Non	Top-2	Requise
COMPLEXITY-DEEP	μ forward	Lexical dur	Non utilisée

7.2 Analyse du routage par token

On pourrait craindre que le routage déterministe (sans perte d'équilibrage de charge) conduise à un déséquilibre des experts. Notre analyse indique que le routage Zipf-balanced garde tous les experts actifs :

- **Utilisation proche de l'équilibre** : dans le run 300M, l'utilisation finale observée des experts est 0,2481/0,2635/0,2481/0,2403, sans expert mort.
- **Normes équilibrées** : les normes des experts restent proches entre branches routées.
- **Pas d'effondrement** : les 4 experts restent différenciés (différence inter-experts $\sim 0,022$)

Spécialisation fonctionnelle des experts : nous mesurons la spécialisation par la *perplexité par expert* plutôt que par la similarité cosinus (qui est un artefact géométrique attendu en haute dimension). Chaque expert est évalué sur son sous-ensemble de tokens assignés et comparé au MLP dense sur les mêmes tokens.

Table 6: Token-Routed MLP vs MoE à routage souple

Critère	Token-Routed	MoE standard	Avantage
Équilibre des experts	Bin-packing par fréquence	Perte aux. requise	Token-Routed
Spécialisation	Fonctionnelle (PPL/expert)	Apprise (softmax)	Dépend
Déploiement	Sans réseau de gating	Gating + dispatch	Token-Routed
Déterminisme	100%	Non (softmax)	Token-Routed
Shared Expert	Oui (patterns communs)	Optionnel	Token-Routed
Type spécialisation	Lexicale	Sémantique	MoE standard

7.3 Ablation de composants (187M, 500M tokens)

Pour isoler la contribution de chaque composant, nous entraînons des modèles **from scratch** sur FineWeb-Edu avec des hyperparamètres identiques (AdamW $\beta = (0.9, 0.95)$, weight decay = 0.1, scheduler cosinus, warmup dynamique 5%, gradient clipping = 1.0, BF16, batch=128). Quatre configurations (~ 171 –187M) sont comparées sur 954 étapes :

Table 7: Configurations d'ablation iso-paramètres (~ 166 –187M, 8B tokens FineWeb-Edu).

Run	Configuration	Params	MLP	Mu	Shared	Experts
run1	Dense SwiGLU baseline	171M	Dense (2416)	—	—	1
run2	Full Complexity	187M	Token-Routed (512/e)	✓	✓	4
run3	TR + Shared + Zipf (sans Mu)	187M	Token-Routed (512/e)	—	✓	4
run4	Mixtral (routeur appris)	187M	Mixtral-style (512/e)	—	—	4

À cette échelle proxy, le modèle complet (Run 2) converge plus vite que la baseline dense ($-0,112$) et que le MoE Mixtral-style ($-0,050$). L'ablation sans Mu (Run 3) est légèrement plus lente que dense à 187M

Table 8: Ablation complète (500M tokens, 954 étapes). La perte moyenne mesure la vitesse de convergence globale.

Configuration	Params	Perte moy.	Δ vs Dense
Run 1 : Dense (SwiGLU)	171M	4,905	—
Run 2 : TR + Shared + Mu + Zipf	187M	4,793	−0,112
Run 3 : TR + Shared + Zipf (sans Mu)	187M	4,916	+0,011
Run 4 : Mixtral (routeur appris, top-1)	187M	4,843	−0,062

(+0,011 en perte moyenne), ce qui suggère que Mu-Guidance aide cette configuration de petite échelle. Cette conclusion ne se transfère pas automatiquement au run 300M, où le meilleur résultat à budget aligné est obtenu par une variante Token-Routed résiduelle sans Mu-Guidance.

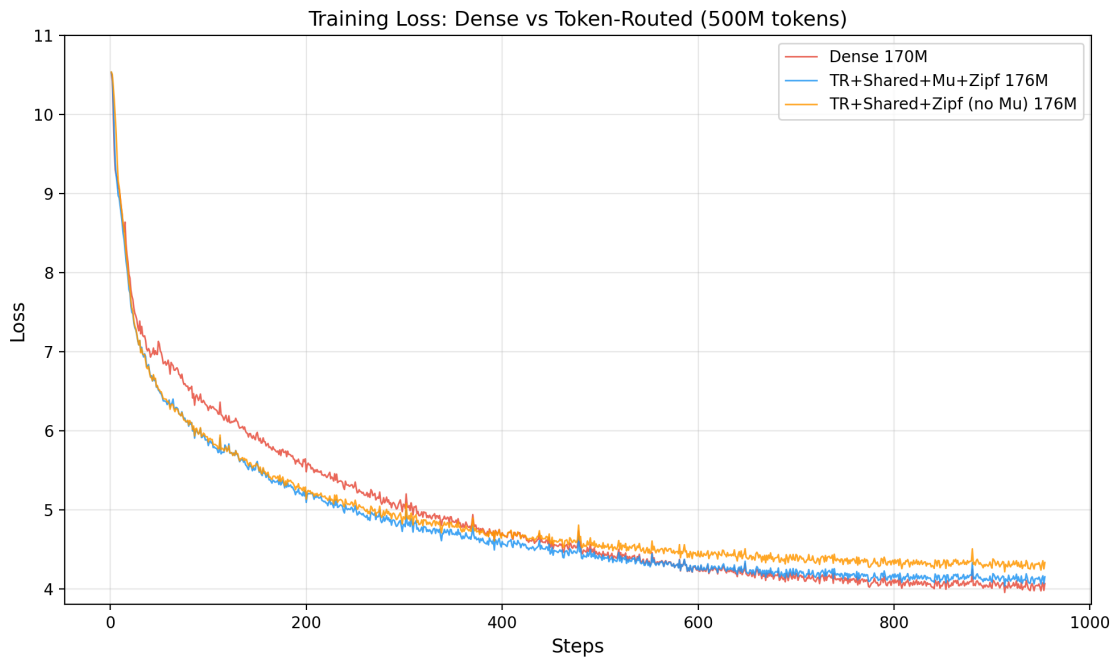


Figure 6: Courbes de perte d’ablation (187M, 500M tokens) : Dense vs Token-Routed complet (Shared+Mu+Zipf) vs Token-Routed sans Mu-Guidance. Le modèle complet (bleu) converge plus vite.

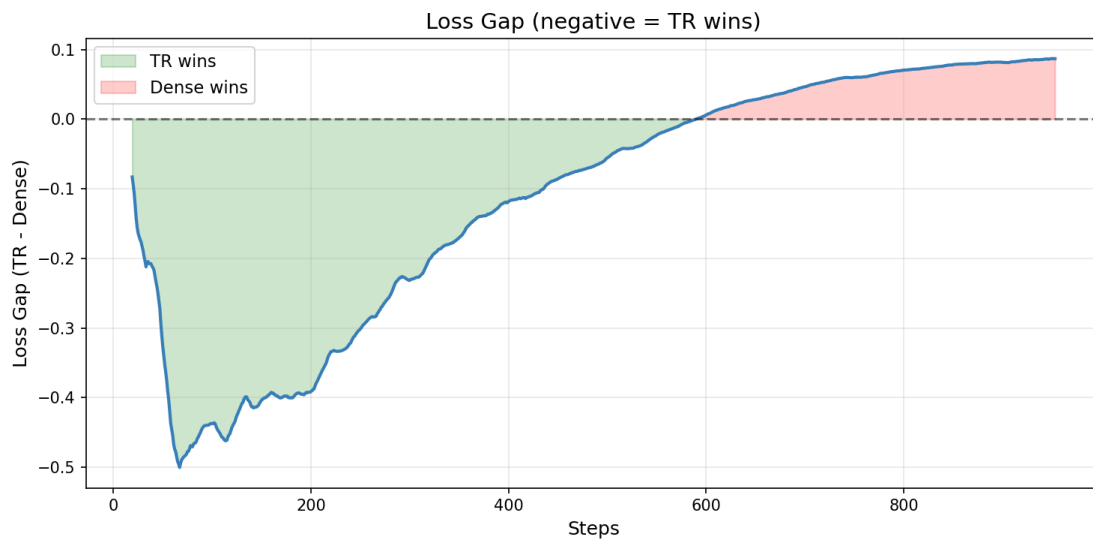


Figure 7: Écart de perte d’ablation TR – Dense (187M, 500M tokens). Négatif (vert) = TR gagne. Le TR mène sur 95% du training.

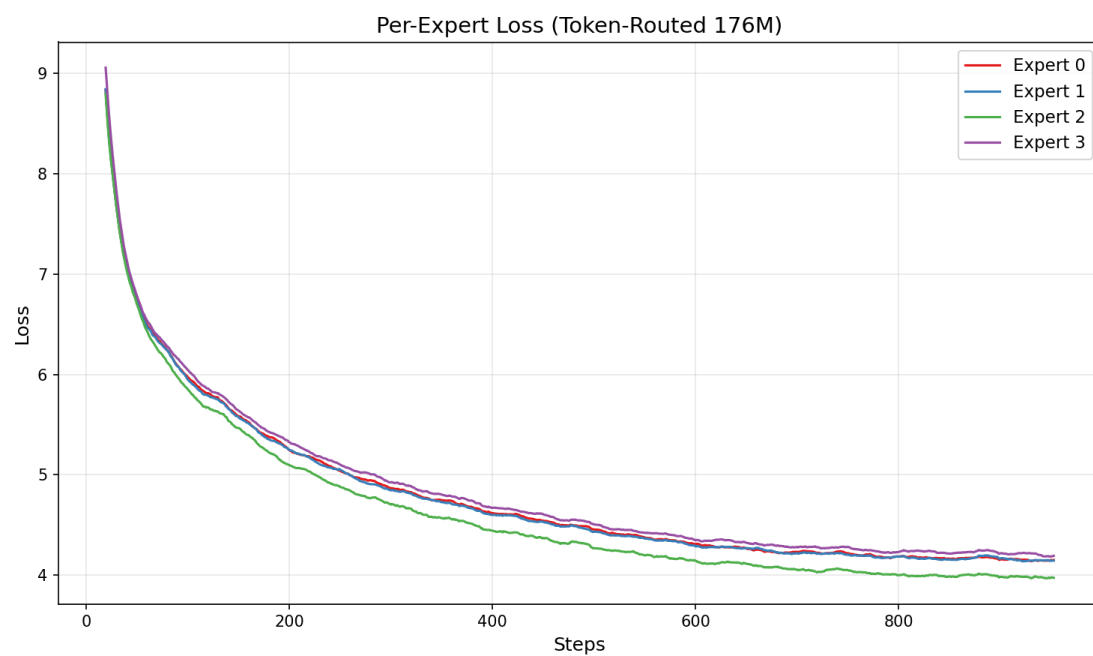


Figure 8: Perte par expert pendant l’entraînement d’ablation (187M). Les 4 experts convergent en parallèle (écart max 0,23).

7.4 Comparaison au scale (300M, 8B tokens)

Pour valider l’architecture corrigée à plus grande échelle, nous entraînons des modèles iso-paramètres (Table 4) sur un budget FineWeb-Edu de 8B tokens. Les deux runs utilisent le même batch global de 1 048 576 tokens/step (8 GPUs $\times$ batch 64 $\times$ séquence 2048) ; numéro de step identique équivaut donc à tokens vus identiques, sans interpolation nécessaire.

Les deux runs partent d’une perte initiale comparable (TR 10,58, dense 10,54) et le modèle Token-Routed est en retard sur la baseline dense durant la phase initiale, le temps que ses experts (encore aléatoires) se spécialisent : au step 100 (≈ 105 M tokens) l’écart est de +0,177 en faveur de dense, avec un pic à +0,31 vers le step 40. L’écart se réduit globalement à mesure que les experts Zipf-routés se spécialisent : le premier gain train-loss loggé apparaît au step 740 (≈ 776 M tokens), et le premier gain en validation alignée apparaît au step 750. L’avance culmine vers $-0,023$ autour du step 2000, puis se réduit légèrement à mesure que les deux runs convergent : au step 7620 ($\approx 7,99$ B tokens, fin du budget) Token-Routed atteint une perte d’entraînement de **2,9231** contre **2,9324** pour dense, et l’écart lissé sur les 50 derniers steps loggés est de $-0,0163$ en faveur de Token-Routed. L’utilisation des experts reste équilibrée (0,2481/0,2635/0,2481/0,2403, zéro expert mort), confirmant que le gain ne vient pas d’un effondrement du routage.

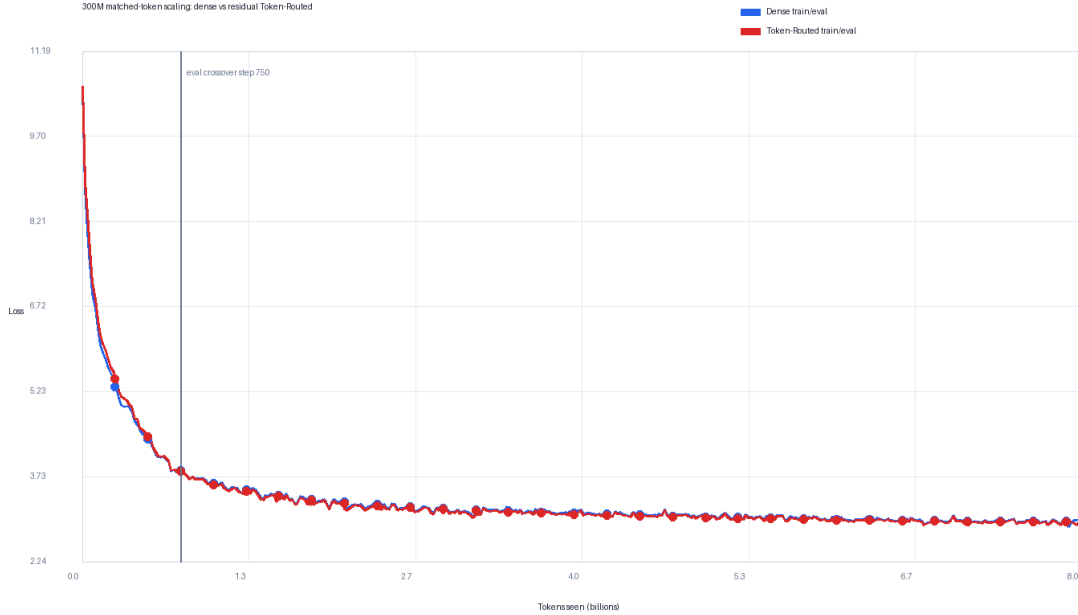


Figure 9: Courbes de scaling 300M corrigées à iso-batch (1 048 576 tokens/step). Token-Routed (rouge) est en retard sur dense (bleu) durant les ≈ 700 premiers steps pendant que ses experts se spécialisent, atteint la parité, puis le crossover : au step 1000 ($\approx 1,05$ B tokens) Token-Routed est devant.

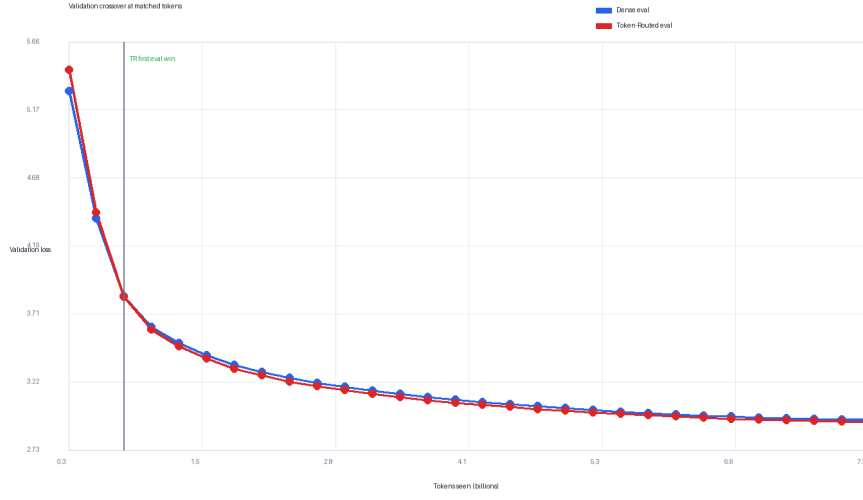


Figure 10: Comparaison validation-loss au même step. Token-Routed paie un coût de spécialisation aux deux premiers points de validation, puis croise la baseline dense au step 750 (≈ 786 M tokens) et reste devant jusqu’au budget 8B tokens.

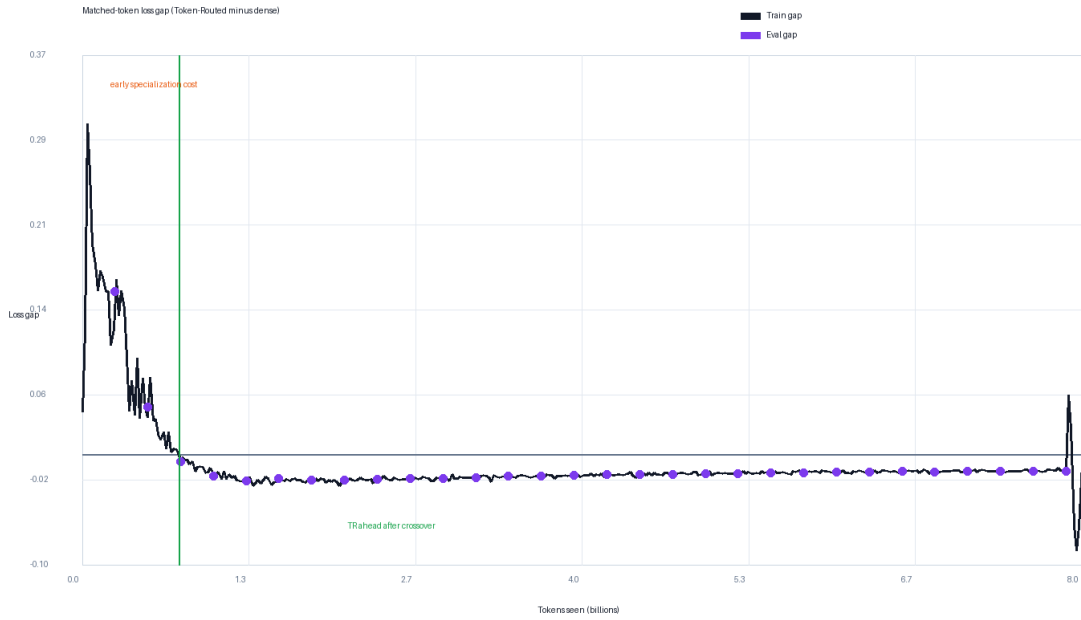


Figure 11: Écart de perte d’entraînement au même step (= tokens vus, iso-batch), calculé comme perte Token-Routed moins perte dense. Les valeurs négatives indiquent que Token-Routed est devant. L’écart est positif (TR derrière) pendant la phase de spécialisation initiale puis devient négatif au step loggé 740.

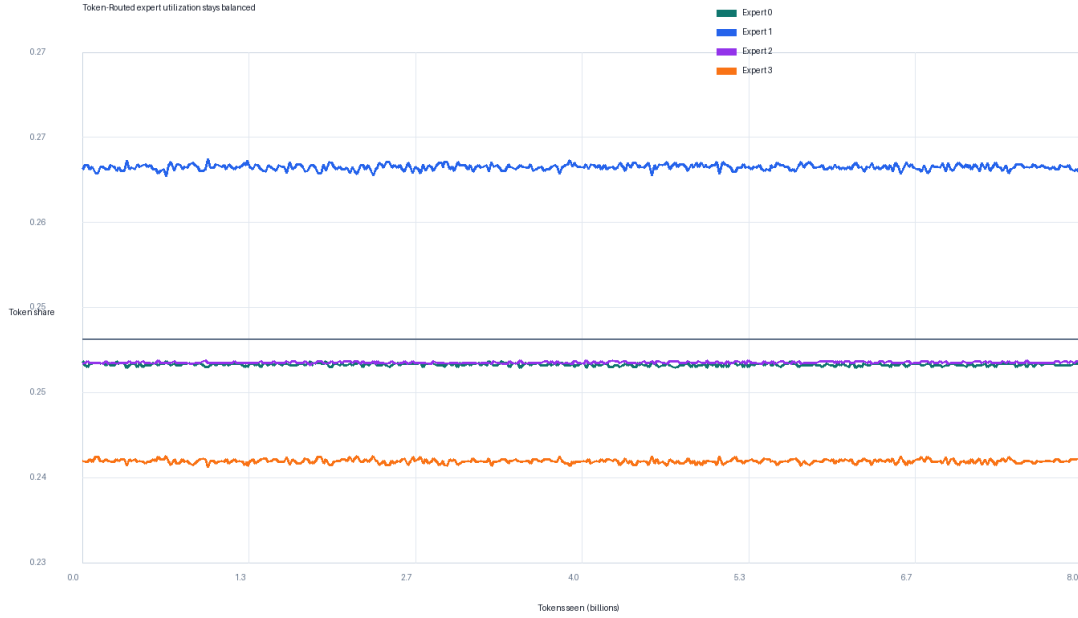


Figure 12: Utilisation des experts pendant le run Token-Routed 300M corrigé. Les quatre experts restent actifs et proches de l’équilibre ; le gain observé ne vient pas d’un effondrement d’expert.

Table 9: Comparaison scaling 300M à iso-batch (1 048 576 tokens/step), run complet 8B tokens. Train-loss au même step. Un écart négatif indique Token-Routed en avance. Dernière ligne : moyenne lissée sur les 50 derniers steps logés.

Step	Tokens vus	Dense (306,5M)	TR (306,5M, k=2)	Écart (TR – Dense)
100	105M	6,6698	6,8464	+0,177
500	524M	4,4450	4,4796	+0,035
700	734M	3,8567	3,8619	+0,005
1000	1,05B	3,5500	3,5324	−0,018
2000	2,10B	3,1953	3,1720	−0,023
4000	4,19B	3,0925	3,0738	−0,019
6000	6,29B	3,0061	2,9906	−0,015
7620	7,99B	2,9324	2,9231	−0,009
lissé 50 derniers	—	2,9446	2,9283	−0,016

Débit d’entraînement. Le modèle Token-Routed 300M utilise une grande branche partagée et une petite branche routée $\text{top-}k = 2$; il n’est donc pas conçu comme un benchmark pur de vitesse $\text{top-}1$. Les deux runs ont été entraînés sur $8 \times \text{B300}$, dense atteignant environ 0,95M tokens/s et Token-Routed environ 0,75M tokens/s au même batch global (1 048 576 tokens/step). Toutes les comparaisons de qualité ci-dessus sont à tokens vus identiques. Le lookup de la table de routage est $O(1)$ et ne requiert pas la synchronisation d’un routeur appris.

7.5 Performance d’inférence

Le modèle Run 2 (187M) déployé sur vLLM 0.18 avec PagedAttention et CUDA graphs atteint un débit soutenu de **8 078 tokens/s** (pic 10 179 tokens/s) en servant 100 requêtes concurrentes sur un seul GPU NVIDIA RTX PRO 6000 (96 Go), avec un TTFT médian (time-to-first-token) de 29,3 ms et une ITL médiane (inter-token latency) de 7,9 ms (Figure 13). Le routage déterministe par token (sans routeur appris) est nativement compatible avec la capture CUDA graph, éliminant les synchronisations CPU–GPU nécessaires dans les architectures MoE classiques. Cette propriété est indépendante de la taille du modèle : contrairement aux MoE à routeur appris qui nécessitent des communications all-to-all entre GPUs, le routage déterministe ne requiert aucune coordination inter-GPU, ce qui suggère un scaling favorable du débit avec le nombre de GPUs et la taille du modèle.

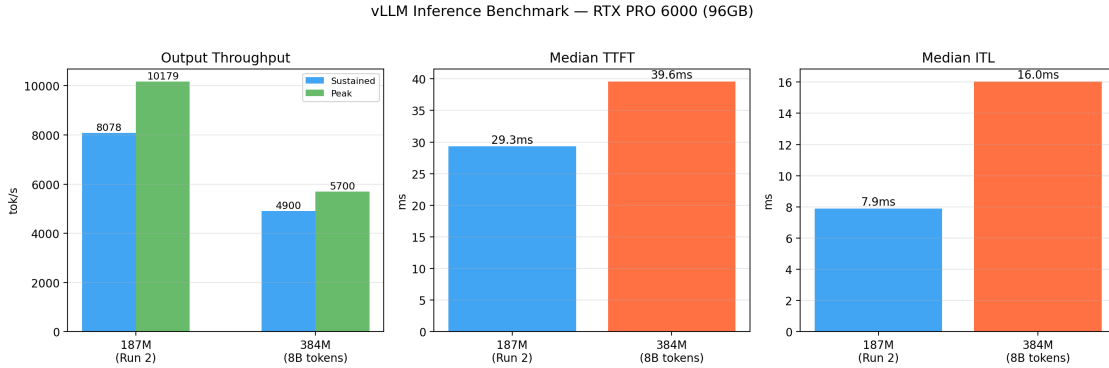


Figure 13: Benchmark d’inférence sur un seul NVIDIA RTX PRO 6000 (96 Go). 100 requêtes à 10 RPS, 128 tokens d’entrée, 1 900 tokens de sortie. Débit soutenu : 8 078 tok/s ; pic : 10 179 tok/s ; TTFT médian : 29,3 ms ; ITL médiane : 7,9 ms.

7.6 Limitations et travaux futurs

- **Résultat de scaling single-seed** : le run 300M corrigé est une comparaison complète à budget aligné sur 8B tokens, mais il reste un seul seed par architecture. Des seeds supplémentaires quantifieraient la robustesse de l’écart lissé final de $-0,0163$ à la variance d’initialisation.
- **Mu-Guidance au scale** : l’ablation 187M soutient Mu-Guidance, mais le gain corrigé à 300M utilise un modèle Token-Routed résiduel sans Mu-Guidance. Un run Mu-guided complet à 300M reste un travail futur.
- **Table de routage spécifique à la distribution** : le bin-packing Zipf utilise une estimation empirique des fréquences de la distribution d’entraînement. Les changements de domaine peuvent modifier l’utilisation réalisée des experts et doivent être mesurés lors du transfert de la table.
- **Benchmarks standardisés** : l’évaluation zero-shot sur ARC-Easy, HellaSwag et MMLU via `lm-evaluation-harness` doit être relancée sur les checkpoints 300M corrigés.

8 Conclusion

Nous avons présenté COMPLEXITY-DEEP, une architecture de modèle de langage avec trois contributions principales : (1) le Token-Routed MLP avec routage Zipf-balanced par bin-packing glouton et Shared Lexical Expert pour une assignation déterministe sans pertes auxiliaires, (2) le Mu-Guided Attention avec μ_{init} apprenable et production de μ après le MLP pour une communication inter-couche tenant compte des experts, et (3) une recette d’entraînement avec warmup dynamique (5%) et initialisation GPT-style ($1/\sqrt{2L}$).

L’ablation de composants à 187M (500M tokens) montre que la configuration complète Token-Routed+Shared+Mu surpasse la baseline dense en perte moyenne (4,793 vs 4,905, $\Delta = -0,112$). Dans le run de scaling iso-paramètres 300M à iso-batch (1 048 576 tokens/step, budget complet 8B tokens), la variante Token-Routed résiduelle sans Mu paie un coût de spécialisation transitoire pendant les ≈ 700 premiers steps (pic +0,31 vers le step 40), gagne pour la première fois en train-loss loggé au step 740 et en validation au step 750, puis reste en avance jusqu’à la fin : l’écart train-loss lissé sur les 50 derniers steps loggés est de $-0,0163$ en faveur de Token-Routed (2,9283 vs 2,9446). Ces résultats soutiennent le routage lexical déterministe comme alternative viable au routage MoE appris à nombre de paramètres aligné, tout en laissant le scaling multi-seed et le scaling Mu-guided complet comme prochaines étapes importantes. L’inférence sur vLLM 0.18 atteint 8 078 tok/s pour le modèle 187M ; les chiffres 300M mis à jour seront reportés depuis le checkpoint corrigé.

Déclaration d’impact sociétal

Ce travail introduit des innovations architecturales pour les modèles de langage. Bien que ces modèles aient de nombreuses applications, ils comportent également des risques de mauvaise utilisation. Nous publions les poids du modèle pour permettre la recherche tout en encourageant une utilisation responsable.

Déclaration de reproductibilité

Pour faciliter la reproductibilité et la révision, nous fournissons l’implémentation complète de l’architecture du modèle en matériel supplémentaire (`supplementary_code.zip`). Le code (PyTorch 2.0+) sera rendu publiquement disponible après acceptation.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022.
- Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pp. 7480–7512, 2023.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.
- Charles D Gilbert and Wu Li. Top-down influences on visual processing. *Nature Reviews Neuroscience*, 14(5):350–363, 2013.
- Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- Rajesh PN Rao and Dana H Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarek, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Roman Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2018.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- George Kingsley Zipf. *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology*. Addison-Wesley Press, 1949.

A Courbes d’entraînement

Des courbes d’entraînement et figures d’analyse supplémentaires sont disponibles dans le matériel supplémentaire.